



PROGRAMMATION ORIENTÉE OBJET

**OBJECTIF: CONCEPTION ET IMPLÉMENTATION EN
JAVA DU SYSTÈME « GESTION DES EMPLOIS DU
TEMPS À L'UASZ »**

Par Ibrahima DIOP

Programmation orientée objet (POO)

INTRODUCTION

Pourquoi la POO

- Pour la **production de programmes** possédant une bonne modularité :
 - ❖ On veut développer une partie d'un programme sans qu'il soit nécessaire de connaître les détails d'implémentation des autres parties
 - ❖ On pourra modifier un module sans que cela n'affecte le reste du programme
- Ainsi faciliter **l'écriture, la maintenance et la réutilisabilité de code.**

Langages de POO

- C++,
- Java,
- C#,
- VB.NET,
- Python,
- PHP (à partir de la version 5),
- Etc.

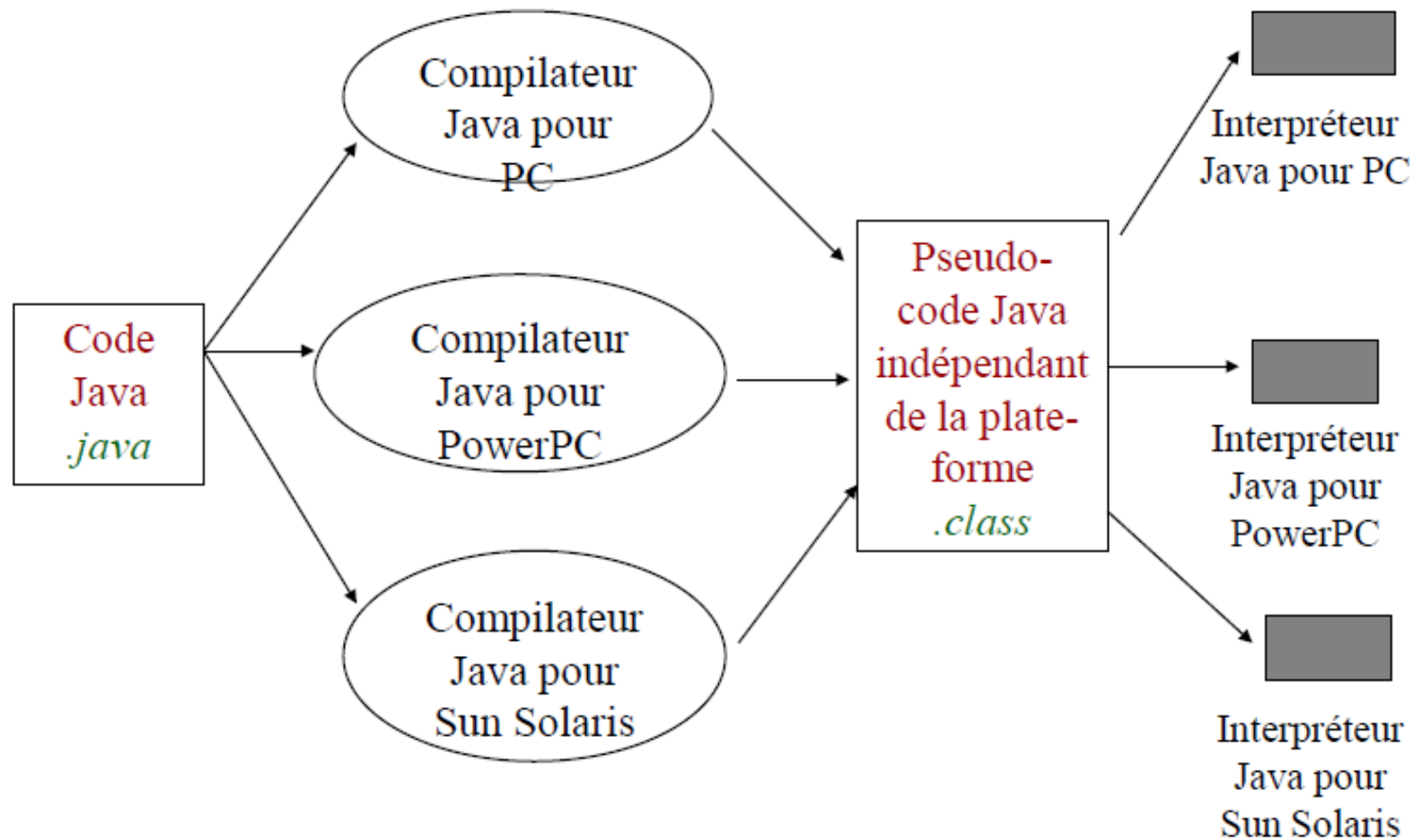
Langages de POO

- C++,
- Java,
- C#,
- VB.NET,
- Python,
- PHP (à partir de la version 5),
- Etc.

C'est quoi Java ?

- Langage orienté objet développé par Sun en 1990.
- Java peut être séparé en deux parties.
 - D'une part, un **programme écrit en langage Java**
 - et d'autre part, une **machine virtuelle (JVM)** qui va se charger de l'exécution de votre programme Java.
- C'est cette plateforme qui garantit la portabilité de Java.

Java est portable



C'est quoi Java ?

- Le Langage Java est dédié à Internet grâce à l'attrait des applets.
- Les **applets** sont des **programmes Java encapsulés dans des pages Web**
 - et qui permettent par exemple l'**animation des pages** ou l'**interactivité entre le serveur et le client Web**.
- Grande richesse des APIs proposées : interfaces graphiques, bases de données, réseau, ...
- Syntaxe proche de C++
- Une grande communauté d'utilisateurs

Kit de développement Java (1)

- **Choix 1** : Le JDK de Sun (oracle.com/java/technologies)
 - Référence pour développer en Java.
 - Comporte tous les outils indispensables à la réalisation et l'exécution d'un programme Java
 - totalement gratuit
 - ne comporte pas de véritable environnement de développement.

Kit de développement Java (2)

- **Choix 2** : Freeware ou shareware offrent une interface graphique au JDK
 - des outils supplémentaires (générateurs d'interface...)
 - Exemple : Eclipse, NetBeans
- **Choix 3** : Environnement de développement professionnel très puissants
 - très chers...
 - IntelliJ
 - *JBuilder 2 de Inprise (anciennement Borland)*
 - *Visual J++ de Microsoft*

Objectif Général

- Vous permettre de concevoir et d'implémenter une solution logicielle modulaire et évolutive avec l'utilisation du paradigme orienté objet.
- **THÈME : CONCEPTION ET IMPLÉMENTATION DU SYSTÈME « GESTION DES EMPLOIS DU TEMPS À L'UASZ »**

Objectifs spécifiques

- **À l'issu de ce cours, vous serez capable:**
 - De modéliser une application selon le concept orienté objet
 - D'écrire les classes identifiées lors de la conception
 - D'intégrer les classes dans un programme exécutable
 - De pouvoir étendre une application existante afin de la faire évoluer (introduction de nouveaux modules, modification de modules existantes, ...)

Pré requis

- Pour suivre ce cours vous devez avoir:
 - Une bonne connaissances de l'algorithmique et de la programmation procédurale

Stratégie pédagogique

- Alternance de **cours théoriques** sous forme d'exposé avec des **Travaux dirigés** (Échanges avec les étudiants) et des **Travaux pratiques**.
- **Ateliers de génie logiciel** pour la conception orientée objet du système de gestion du matériel dans l'UASZ.

Partie 1: Rappel des Concepts et fondamentaux de la POO en JAVA

PLAN

- **Syntaxe de base de JAVA**
 - Les structures de contrôle
 - Tableaux / Types énumérés
 - Les types et les Entrées / Sorties simplifiées
- **Fondements de la POO en JAVA**
 - Classes et objets
 - Package
 - Visibilité des classes, variables et méthodes
 - Héritage & Polymorphisme
 - Classes abstraites et interfaces
- **Utilisation de bibliothèques JAVA**
 - Exceptions
 - Interfaces graphiques avec Javafx
 - Persistance avec JDBC

Partie 1: Rappel des Concepts et fondamentaux de la POO

PLAN

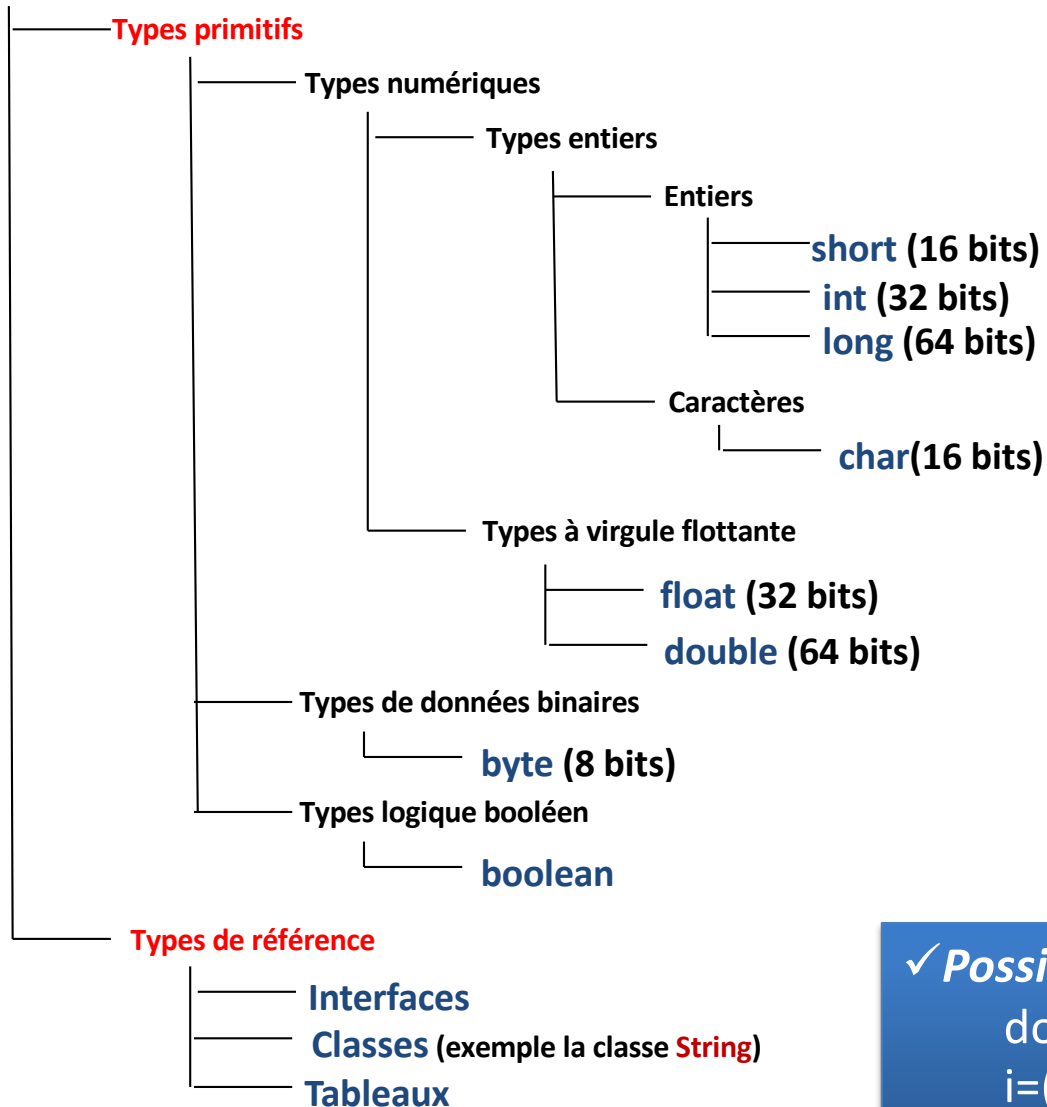
- **Syntaxe de base du langage JAVA**
 - Les structures de contrôle
 - Tableaux / Types énumérés
 - Les types et les Entrées / Sorties simplifiées
- **Fondements de la POO en JAVA**
 - Classes et objets
 - Package
 - Visibilité des classes, variables et méthodes
 - Héritage & Polymorphisme
 - Classes abstraites et interfaces
- **Utilisation de bibliothèques JAVA**
 - Exceptions
 - Interfaces graphiques avec Javafx
 - Persistance avec JDBC

Les commentaires

- Les commentaires tiennent sur une ligne :
`// tout le reste de la ligne est un commentaire`
- Les commentaires sont multi-lignes :
`/* ceci est un commentaire
tenant sur deux lignes
*/`
- Les commentaires sont destinés au système **javadoc** pour la génération d'une documentation API à partir du code :
`/** ceci est un commentaire spécial destiné au
système JavaDoc */`

Les types de données JAVA

Types de données Java



✓ Possibilité de faire du casting comme en C

```
double d; int i;  
i=(int)d; //troncation de d en un entier
```

La classe String

- **String n'est pas un type primitif, c'est une classe**
- **Déclaration de deux String:**

```
String s1, s2;
```

- **Initialisation :**

```
s1 = 'Hello';
```

```
s2 = 'le monde';
```

- **Déclaration et initialisation :**

```
String s3 = 'Hello';
```

- **Concaténation :**

```
String s4 = s1 + ' ' + s2;
```

Programmation orientée objet (POO)

Syntaxe de base du langage JAVA

LES STRUCTURES DE CONTRÔLE

Instructions conditionnelles (IF)

Syntaxe:

```
if (condition)
{
    instruction1
}
else
{
    instruction2
}
```

Exemple:

```
If ( x > 0)
{
    System.out.println('positif ');
    x-- ;
}
else
    System.out.print('négatif ');
```

Instructions itératives (Pour...Faire)

Syntaxe:

```
for (initialisation; condition; expression)
{
    instructions
}
```

Exemple:

```
for (int i=0 ; i<5 ; i++)
    System.out.print('Salut!');
```

Instructions itératives (TantQue...Faire...)

Syntaxe:

```
while (condition)
{
    instructions
}
```

Exemple:

```
int i = 10 ;
while ( i>0)
{
    System.out.println(i);
    i-- ;
}
```

Instructions itératives (Faire...TantQue...)

Syntaxe:

```
do
{
  instructions
}
while (condition);
```

Exemple:

```
do
  a++
while (a != b);
```


L'instruction Switch

Syntaxe:

```
switch (expression)
{
    case valeur1:
        instructions; break;
    case valeur2:
        instructions; break;
    ...
    default:
        instructions
}
```

Exemple:

```
switch (i) {
    case 10 : System.out.println('x
        égale 10 '); break ;
    case 100 : System.out.println('x
        égale 100 '); break ;
    default : System.out.println('x est
        différent de 10 et de 100 ');
}
```

Programmation orientée objet (POO)

Syntaxe de base du langage JAVA

TABLEAUX / TYPES ÉNUMÉRÉS

Déclaration des tableaux

- Les tableaux se comportent un peu comme des objets.
- Deux syntaxes pour la déclaration:
- **TypeDesElements**[] **NomDuTableau**;
- **TypeDesElements** **NomDuTableau**[];
- *Exemple:*
 - `int[ ] tab_entiers; ou int tab_entiers[ ];`
 - `String[ ] tab_chaines;`
- Le type des éléments du tableau peut être une classe existante.

Instanciation d'un tableau

- *InstanciationTypeDesElements[]*
NomDuTableau = new
TypeDesElements[TailleDuTableau];
- *Exemple*
 - int[] tab_entiers = new int[15];
 - String[] chaines = new String[3];

Exemple de saisie d'un tableau

```
public static void main(String[] args) {  
    int[] t = new int[5];  
    for (int i=0; i<5; i++){  
        t[i] = Integer.parseInt(args[i]);  
    }  
}
```

Matrices

- TypeDesElements[][] NomDuTableau;
- TypeDesElements NomDuTableau[][];
- *Exemple*
 - int[][] matrice
 - *ou int matrice[][];*

Déclaration d'un type énuméré (1/2)

- qualifieurs enum identif { identif, identif, ... identif }
- *Exemples :*
 - public enum PointCardinal { NORD, SUD, EST, OUEST }
 - public enum JourSemaine { LUNDI, MARDI, MERCREDI, JEUDI, VENDREDI, SAMEDI, DIMANCHE }

Déclaration d'un type énuméré (2/2)

- Ces déclarations créent 2 nouveaux types publics **PointCardinal** et **JourSemaine**.
- Elles doivent être écrites dans 2 fichiers séparés, respectivement nommés *PointCardinal.java* et *JourSemaine.java*.

Programmation orientée objet (POO)

Syntaxe de base du langage JAVA

TYPES ET ENTRÉES / SORTIES

La méthode printf

- La classe **java.io.PrintStream** contient une méthode **printf** qui fonctionne presque de la même façon qu'en C.
- **Exemple** :
 - `int degre = 28;`
 - `System.out.printf('Il fait %d degres\n', degre);`
 - `// %d` indicateur de format pour des entiers
- Comme en C on utilise des **indicateurs de format** commençant par le symbole **%**.

javax.swing.JOptionPane

- String **entree** =
JOptionPane.showInputDialog("demande de données d'entrées");
- int **sortie1**=Integer.parseInt(**entree**);
- ...
- int **sortie2** = 2***sortie1**;
- ...
- **JOptionPane.showMessageDialog**(null, "la sortie 1 est "+**sortie1**);
- **System.out.println**("la sortie 2 est "+**sortie2**);

La classe java.util.Scanner

```
Scanner entree = new Scanner(System.in);  
System.out.println("Donner votre prenom et votre nom:");  
String prenom = entree.next(); String nom = entree.next();  
System.out.println("Donner votre age:");  
int age = entree.nextInt();  
entree.nextLine();  
System.out.println("Donner une phrase:");  
String phrase = entree.nextLine();  
System.out.printf("%s %s, %d ans, dit %s\n", prenom, nom, age,  
phrase);
```

La classe `java.util.Scanner`

Offre des méthodes pour **lire** un **flux de données** formatées :

- **`int nextInt()`** : lecture de la prochaine valeur de type `int`.
- **`long nextLong()`** : lecture de la prochaine valeur de type `long`.
- **`float nextFloat()`** : lecture de la prochaine valeur de type `float`.
- **`double nextDouble()`** : lecture de la prochaine valeur de type `double`.
- **`String nextLine()`** : lecture de tous les caractères se présentant dans le flux d'entrée jusqu'à une marque de fin de ligne et renvoi de la chaîne ainsi construite. **La marque de fin de ligne est consommée, mais n'est pas incorporée à la chaîne produite.**

La classe java.util.Scanner

Offre aussi toute une série de **méthodes booléennes** pour tester le type de la prochaine donnée disponible, sans lire cette dernière :

- ✓ **boolean hasNext()** : Y a-t-il une donnée disponible pour la lecture
- ✓ **boolean hasNextLine()** : Y a-t-il une ligne disponible pour la lecture
- ✓ **boolean hasNextInt()** : La prochaine donnée à lire est-elle de type int
- ✓ **boolean hasNextLong()** : La prochaine donnée à lire est-elle de type long
- ✓ **boolean hasNextFloat()** : La prochaine donnée à lire est-elle de type float
- ✓ **boolean hasNextDouble()** : La prochaine donnée à lire est-elle de type double

Partie 1: Rappel des Concepts et fondamentaux de la POO

PLAN

- Syntaxe de base du langage JAVA
 - Les structures de contrôle
 - Tableaux / Types énumérés
 - Les types et les Entrées / Sorties simplifiées
- **Fondements de la POO en JAVA**
 - **Classes et objets**
 - **Package**
 - **Visibilité des classes, variables et méthodes**
 - **Héritage & Polymorphisme**
 - **Classes abstraites et interfaces**
- **Utilisation de bibliothèques JAVA**
 - **Exceptions**
 - **Interfaces graphiques avec Javafx**
 - **Persistance avec JDBC**